

Actividad Práctica: Detección de Spam con Machine Learning

Metodología

Aprendizaje basado en proyectos + práctica guiada

Objetivos de Aprendizaje

- Comprender qué es un sistema de clasificación.
- Preparar datos de texto para Machine Learning.
- Entrenar un modelo simple de detección de spam.
- Evaluar resultados.
- Aplicar IA a un problema real.

Las empresas reciben miles de correos diarios. Muchos son spam. Vamos a crear un **sistema automático** que decida si un mensaje es:

-  Legítimo
-  Spam

usando Machine Learning.

PARTE 1 – Preparación del entorno

Paso 1. Instalar herramientas

- Python 3
- `pip install pandas scikit-learn nltk`

Paso 2. Descargar dataset

Usa el dataset clásico:  **SMS Spam Collection** o crea el tuyo propio

Formato:

Spam, Win money now!!!

ham, Hola qué tal?

Archivo: spam.csv

PARTE 2 – Cargar y explorar datos

Paso 3. Importar librerías

```
import pandas as pd
```

Paso 4. Cargar datos

```
data = pd.read_csv("spam.csv", sep="\t", names=["label", "message"])
```

Paso 5. Ver datos

```
data.head()
```

- ¿Qué valores tiene label?
- ¿Cuántos mensajes hay?

```
data["label"].value_counts()
```

PARTE 3 – Preprocesamiento del texto

Paso 6. Convertir texto en números

Los ordenadores no entienden palabras → necesitan números, así que usa **TF-IDF**.

```
from sklearn.feature_extraction.text import TfidfVectorizer
```

Paso 7. Crear vectorizador

```
vectorizer = TfidfVectorizer(stop_words="english")
```

```
X = vectorizer.fit_transform(data["message"])
```

Paso 8. Crear etiquetas

```
y = data["label"].map({"ham": 0, "spam": 1})
```

PARTE 4 – Entrenamiento del modelo

Paso 9. Separar datos

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

Paso 10. Elegir algoritmo

Usa **Naive Bayes** (ideal para texto).

```
from sklearn.naive_bayes import MultinomialNB
```

Paso 11. Entrenar modelo

```
model = MultinomialNB()
```

```
model.fit(X_train, y_train)
```

PARTE 5 – Evaluación

Paso 12. Predicciones

```
y_pred = model.predict(X_test)
```

Paso 13. Medir precisión

```
from sklearn.metrics import accuracy_score  
accuracy_score(y_test, y_pred)
```

Interpretación:

- 0.90 → Muy bueno
- 0.80–0.90 → Aceptable
- < 0.80 → Mejorable

Paso 14. Informe completo

```
from sklearn.metrics import classification_report  
print(classification_report(y_test, y_pred))
```

PARTE 6 – Probador de mensajes

Paso 15. Crear función predictora

```
def detectar_spam(texto):  
    vector = vectorizer.transform([texto])  
    pred = model.predict(vector)  
  
    if pred[0] == 1:  
        return "🚫 SPAM"  
    else:  
        return "✅ NO SPAM"
```

Paso 16. Probar mensajes

```
detectar_spam("Win 1000€ now, click here")  
detectar_spam("Hola, ¿quedamos mañana?")
```

PARTE 7 – Reflexión (15 min)

1. ¿Por qué hay errores?

2. ¿Qué palabras influyen más?
3. ¿Podría engañarse al sistema?
4. ¿Qué riesgos éticos existen?

Producto Final

- ✓ Código
- ✓ Capturas de resultados
- ✓ Conclusiones (1 página)

Rúbrica Básica (Resumen)

Criterio	Excelente (10)	Bien (7)	Básico (5)	Insuficiente
Código	Funciona completo	Errores menores	Incompleto	No funciona
Modelo	>90% precisión	>80%	>70%	<70%
Análisis	Profundo	Adecuado	Superficial	Ausente
Autonomía	Alta	Media	Baja	Nula